

The catalogue you imported in the first session describes items and villagers. This sheet adds what is yours: an **island**. Players name their island, villagers move in and out, items pile up in the inventory, and other players come to visit. You model that data as documents, write the data access in PHP with the native driver, query it, and answer two report questions with a pipeline. Everything you need was done in a smaller form in the sessions *Data that Hurts in SQL*, *Document Modelling*, *CRUD and Queries* and *Aggregation I*.

The domain. An island has a name, the player who owns it, a hemisphere (north or south), a creation date and, if known, a native fruit. **Residents:** at most ten villagers of the catalogue live on an island; the island page always shows their names. **Inventory:** items of the catalogue with a count, a few hundred entries at most, changed many times a day. **Visits:** every time another player visits, the game records who came and when, and keeps that forever.

Submission Requirements:

- Read the `Organizational Guidelines` on Moodle, and adhere to them
 - Submit a PDF with, per task, your solution idea, the code, and the test (console output or a Compass screenshot). Name and time spent on the cover page
 - Submit a ZIP with your PHP files. They must run inside the course stack from the folder `app/a01/` with `require __DIR__ . '/../connect.php'`; and nothing else installed; we run `docker compose exec php php a01/demo.php`
 - Naming: `MDB3_A01_LASTNAME.pdf` and `MDB3_A01_LASTNAME.zip`
 - Native driver only (`MongoDB\Client`, as in `connect.php`); no ODM, no framework
-

Exercise 1 (6 points) Model the Island

Design the documents. Deliver:

- One JSON example of an island document with two residents and two inventory entries, and of every other document type your design needs
- For **residents**, **inventory** and **visits**: embed or reference? Answer the four questions of *Document Modelling* in one sentence each (read with the parent? how many, and bounded? shared? changes with the parent?) and state your decision
- What is the `_id` of an island, and why? Two sentences

The residents are villagers of the catalogue and the inventory holds items of the catalogue: say what you copy from the catalogue into your documents and what you only reference.

(A02 continues with these islands. It ships a reference model to start from, so a different decision here costs nothing later.)

Exercise 2 (10 points) The Data Access Layer

Implement your model as two classes with the native driver: `IslandDao` and `VisitDao`, in a database of your own (a01), plus a script `demo.php` that uses every method and prints what happened. Required methods:

- `create(...)`: inserts an island and returns its `_id`
- `findByName(string)`: ?object and `findByResident(int villagerId)`: array: which islands does this villager live on?
- `moveIn(name, villagerId, villagerName)`: bool: a villager moves in; **at most ten residents, and nobody twice**. Returns whether it happened. One update, no read before it
- `moveOut(name, villagerId)`: bool
- `addToInventory(name, itemId, itemName, count)`: void: if the island already has the item, **add to its count**; otherwise append a new entry. No duplicates. (The positional operator `$` in an update path, `"inventory.$.count"`, means “the array element the filter matched”; see the manual, *Update Operators*.)
- `rename(old, new)`: bool
- `delete(name)`: bool: deletes the island **and everything that refers to it**
- `VisitDao::record(islandId, visitor, DateTime)` and `VisitDao::latest(islandId, n)`: the newest `n` visits. Store the time as a `MongoDB\BSON\UTCDateTime`

`demo.php` must: create at least three islands, move at least eleven villagers into one of them (the eleventh must be refused) and one of them out again, add the same item twice to one inventory, record at least twenty visits spread over at least two months, and print the results of every call. Your **test** is the console output, pasted into the PDF.

Exercise 3 (4 points) Four Queries

Write these four as `find()` filters, in mongosh or in PHP, against your own data. For each: the filter, the result, and the SQL you would have written in *Relational Databases* (one line; for the ones that need a junction table, say so instead).

1. All islands where villager 1 (Admiral) lives, names only
2. All islands whose inventory has **at least three** clay furnaces (item 717)
3. The five newest visits to one island in October 2026, newest first
4. All islands that are in the southern hemisphere **or** have no native fruit recorded

Dates in a filter are `UTCDateTime` objects in PHP and `ISODate("2026-10-01")` in mongosh.

Exercise 4 (4 points) Two Reports

Two questions on the visits, each as one aggregation pipeline (mongosh or PHP), with the output and the SQL you would have written:

1. Per island: the number of visits in October 2026 and the time of the last one, busiest island first
2. Visitors who came more than four times in total, most frequent first